

Aperture K2.2 推进任务书

你是一名资深 Java 软件架构师、Minecraft Fabric 开发者、CAD/BIM 内核工程师、实时交互系统工程师和多人同步架构师。

你正在参与开发：

Aperture

Aperture 不是一个 Minecraft 门窗模组。

Aperture 是一个运行在 Minecraft 世界中的：

Architectural Design and Runtime Kernel

它不仅负责生成建筑几何，还负责定义建筑对象的状态、行为、交互、运动、持久化、多人同步和模拟接口。

Aperture 的目标不是复制 Revit、Rhino、Blender 或 AutoCAD，而是建立一种这些软件通常不提供的能力：

让建筑对象从静态模型变成可运行、可交互、可协作、可模拟、可扩展的 Runtime Object。

当前仓库：

<https://github.com/Theopote/aperture>

在开始修改代码前，必须完整检查当前仓库的模块、文档、测试和最近实现。

不要假设本文档中提到的类一定不存在。

如果仓库中已经存在相应实现，应优先：

- 复用；
- 补全；
- 重构；
- 统一；
- 删除重复路径；

而不是创建第二套实现。

一、当前阶段目标

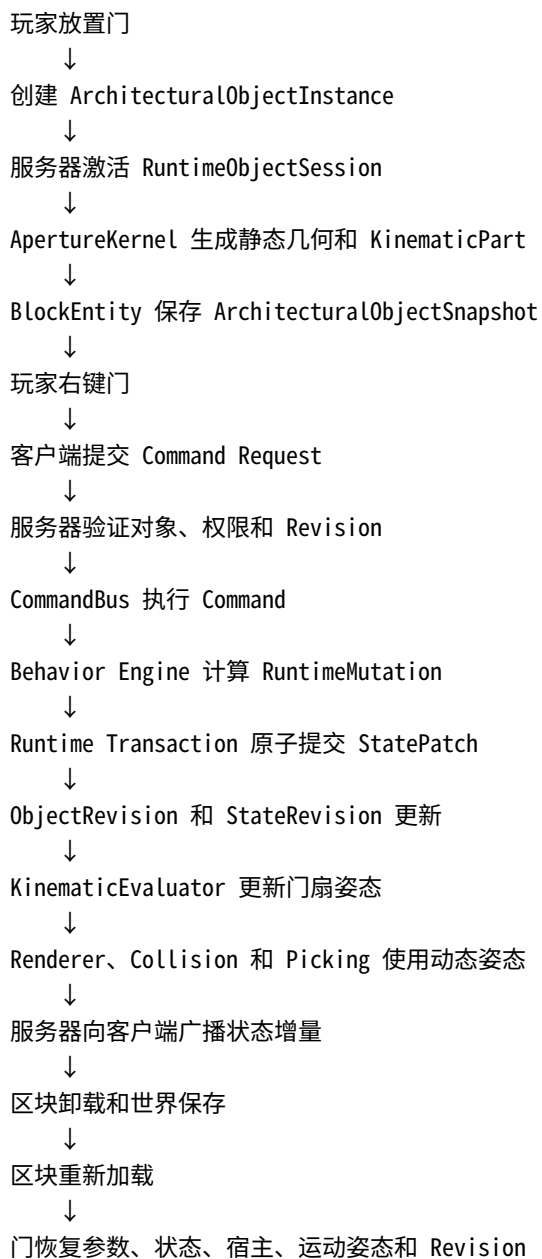
当前阶段命名为：

K2.2 — Server-Authoritative Runtime Vertical Slice

本阶段只选择一个现有参考对象：

aperture:door

完成以下真实 Minecraft 生命周期：



本阶段完成后，Aperture 必须证明：

它生成的不是一份静态门 Mesh，而是一个服务器权威、可持久化、可同步、可运行的建筑对象。

二、严格范围

本阶段必须完成：

1. 统一的 `ArchitecturalRuntime`。
2. `RuntimeObjectSession` 生命周期。
3. `Command` → `Behavior` → `Mutation` → `Transaction` → `State` 的唯一写入链。
4. Door Runtime 完整运行。
5. 服务器权威命令协议。
6. Revision 校验、冲突拒绝与重新同步。
7. 门扇 Kinematic Transform 接入渲染。
8. 动态碰撞接入。
9. 动态 Picking 或选择边界接入。
10. BlockEntity 持久化。
11. 世界保存、卸载、重载闭环。
12. 双客户端状态复制测试。
13. 端到端测试和架构检查。
14. 更新文档和真实实施状态。

本阶段暂时不要实现：

- 新门型；
- 新窗型；
- 新幕墙类型；
- Behavior Graph 可视化编辑器；
- Component Graph 可视化编辑器；
- AI 建筑生成；
- NodeCraft 集成；
- IFC 导入导出；
- 日照、能耗、人流等正式模拟器；
- 大型 Asset Browser；
- 完整 ImGui 编辑器；
- 复杂协同合并算法；
- 精确三角形动态碰撞；
- 每帧重新生成几何。

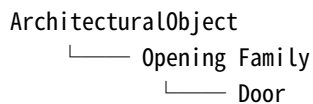
不要扩大范围。

三、架构铁律

整个开发必须遵守以下规则。

1. ArchitecturalObject 是通用运行时原语

正确关系：



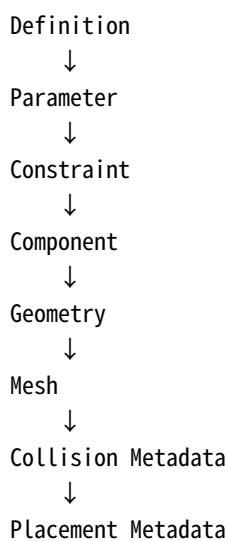
不允许 Runtime Kernel 将 Door 或 Opening 视为唯一可运行对象。

Door 只是第一个完整参考 Family。

2. Generation 与 Runtime 必须分离

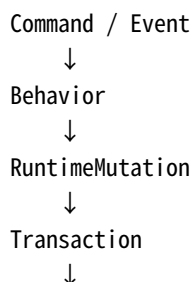
Generation Pipeline 回答：

对象长什么样？



Runtime Pipeline 回答：

对象如何运行？



```
State Transition
↓
Kinematics / World Effects
↓
Persistence / Replication
```

不要把两条 Pipeline 混合。

3. Runtime State 只能通过事务修改

禁止以下写法：

```
state.put("openRatio", value);
session.setState(...);
door.setOpenRatio(...);
```

除 Runtime Transaction 内部外，任何代码都不能直接修改正式状态。

正确方式：

```
Command
↓
CommandHandler
↓
Behavior
↓
RuntimeMutation
↓
RuntimeTransaction.commit()
```

4. Behavior 不能直接修改 Minecraft World

Behavior 只能返回：

```
StatePatch
ArchitecturalEvent
WorldEffect
Diagnostic
ScheduledCommand
```

Fabric Adapter 负责执行 `WorldEffect`。

禁止在 `aperture-runtime-model`、Behavior 或 Command Handler 中导入：

```
net.minecraft.*
```

5. 客户端不能修改权威状态

客户端只能发送：

```
CommandRequest
```

客户端不能发送：

```
openRatio = 1  
locked = false  
state delta
```

服务器负责：

- 校验；
- 执行；
- 提交；
- 增加 Revision；
- 广播结果。

6. 动态运动不能重新生成几何

开门过程中禁止重复执行：

```
Parameter  
→ Geometry  
→ Mesh
```

应执行：

```
RuntimeState.openRatio  
↓  
KinematicEvaluator  
↓  
Door Panel Transform
```

门框、门套等静态部分保持缓存。

7. ImGui 不进入 Kernel

本阶段可为后续 ImGui 保留 Command API，但不要让以下模块依赖 ImGui：

```
aperture-runtime-model  
aperture-core  
aperture-geometry  
aperture-pipeline  
aperture-kernel  
aperture-runtime  
aperture-opening
```

四、开始工作前的仓库检查

第一步不要写代码。

先检查并输出：

```
docs/refactoring/k2-2-runtime-vertical-slice-audit.md
```

文档必须包含：

1. 当前模块

检查至少：

```
aperture-runtime-model  
aperture-runtime  
aperture-opening  
aperture-kernel  
aperture-render  
aperture-editor  
aperture-fabric
```

2. 当前已有类型

检查是否已经存在：

```
ArchitecturalRuntime  
RuntimeObjectSession  
RuntimeState  
StatePatch  
CommandBus
```

```
CommandTransaction
BehaviorEngine
RuntimeMutation
KinematicEvaluator
ArchitecturalObjectSnapshot
ReplicationMessage
CommandRequestMessage
ObjectSnapshotMessage
StateDeltaMessage
EventDeltaMessage
OpeningBlockEntity
```

3. 当前真实调用链

必须追踪：

玩家右键门

从 Fabric 入口开始，一直追踪到：

- Command;
- Runtime;
- Behavior;
- State;
- Replication;
- Render;
- Collision;
- BlockEntity。

不能只看接口定义。

4. 识别重复实现

检查是否存在：

- 两套 CommandBus;
- 两套 Runtime State;
- 两套 Snapshot Codec;
- 两套 Door State Driver;
- 两套 Network Codec;
- BlockEntity 与 Runtime 各自保存状态;
- Behavior 和 Handler 都直接修改状态;
- Client 和 Server 各自推导权威状态。

5. 输出迁移表

格式：

能力	当前实现	目标实现	动作
Runtime session	当前类	统一类	保留/重构/删除
State mutation	当前路径	Transaction	迁移
Door interaction	当前路径	CommandRequest	迁移
Persistence	当前路径	Snapshot	补全
Replication	当前路径	Server authority	补全

完成审计后再开始代码实现。

五、任务一：实现统一 ArchitecturalRuntime

建立或完善：

```
public interface ArchitecturalRuntime {
    RuntimeObjectSession create(ArchitecturalObjectInstance instance);

    RuntimeObjectSession restore(ArchitecturalObjectSnapshot snapshot);

    CommandResult submit(CommandEnvelope envelope);

    TransactionResult submit(CommandTransaction transaction);

    void tick(RuntimeTickContext context);

    void unload(ArchitecturalObjectId objectId);

    void remove(ArchitecturalObjectId objectId);

    Optional<RuntimeObjectSession> find(ArchitecturalObjectId objectId);

    Collection<RuntimeObjectSession> activeObjects();
}
```

默认实现建议：

```
DefaultArchitecturalRuntime
```

它负责协调：

```
RuntimeObjectRepository
CommandBus
BehaviorEngine
```

```
Persistence
ReplicationSink
WorldQueryExecutor
WorldEffectExecutor
KinematicEvaluator
RuntimeDiagnostics
```

它不是 Service Locator。

它必须拥有清晰的对象生命周期。

RuntimeObjectSession

建立：

```
public interface RuntimeObjectSession {
    ArchitecturalObjectInstance instance();

    RuntimeState state();

    CapabilitySet capabilities();

    BehaviorSet behaviors();

    KinematicModel kinematics();

    long objectRevision();

    StateRevision stateRevision();

    long eventSequence();

    DirtyFlags dirtyFlags();

    ArchitecturalObjectSnapshot snapshot();
}
```

实现可命名：

```
DefaultRuntimeObjectSession
```

Session 必须是服务器内一个活动建筑对象的唯一权威内存表示。

不要让 BlockEntity 成为第二个权威状态容器。

BlockEntity 只负责：

- 保存 Snapshot;
- 提供 Minecraft 锚点;
- 在加载时激活 Session;
- 在卸载或保存时提取 Snapshot。

六、任务二：统一状态修改事务

建立或完善以下类型：

```
RuntimeMutation  
RuntimeTransaction  
RuntimeCommitResult  
StateTransition  
CommittedRuntimeChange
```

建议：

```
public record RuntimeMutation(  
    StatePatch statePatch,  
    List<ArchitecturalEvent> events,  
    List<WorldEffect> worldEffects,  
    List<ArchitecturalCommand> scheduledCommands  
) {}
```

事务提交步骤：

1. 查找 RuntimeObjectSession
2. 校验 expectedObjectRevision
3. 校验 expectedStateRevision
4. 校验 StatePatch 是否符合 StateSchema
5. 创建候选状态
6. 校验能力和行为约束
7. 原子提交
8. objectRevision + 1
9. stateRevision + 1
10. eventSequence 更新
11. 计算 DirtyFlags
12. 产生 CommittedRuntimeChange
13. 发送 Replication Delta
14. 标记 Persistence Dirty
15. 执行允许的 WorldEffect

任何失败都不得产生部分修改。

Revision 规则

必须明确：

ObjectRevision

表示对象整体权威版本，包括：

- 参数；
- Transform；
- Host；
- Behavior Configuration；
- Persistent State；
- 删除状态。

StateRevision

只表示 Runtime State 的版本。

EventSequence

表示复制事件的有序序列。

不要混用。

七、任务三：整理 Command 与 Behavior 职责

Command

Command 表达意图，例如：

RequestOpenCommand
RequestCloseCommand
ToggleOpenCommand
SetLockCommand
SetParameterCommand
AttachToHostCommand
TransformObjectCommand

Command 不直接修改状态。

CommandHandler

CommandHandler 负责：

- 基础校验；
- 权限校验；
- Capability 检查；
- 生成 Behavior Trigger 或 RuntimeMutation 请求。

Behavior

Behavior 负责：

- 根据当前状态；
- 参数；
- Capability；
- World Query；
- Event；

决定：

```
StatePatch  
WorldEffect  
ArchitecturalEvent  
ScheduledCommand
```

Door 运行逻辑

例如打开门：

```
RequestOpenCommand  
↓  
Openable Capability 检查  
↓  
DoorOpenBehavior  
↓  
targetOpenRatio = 1.0  
motion = OPENING
```

Runtime Tick：

```
motion = OPENING  
↓  
根据 deltaTime 和 openingSpeed  
↓  
更新 openRatio  
↓
```

```
到达 1.0
↓
motion = IDLE
```

关闭门同理。

锁定状态下：

```
RequestOpenCommand
↓
CommandRejected
↓
DiagnosticCode.OBJECT_LOCKED
```

八、任务四：实现 Door Runtime Lifecycle

确保 Door 的 State Schema 至少包含：

```
openRatio
targetOpenRatio
motion
locked
enabled
lastInteractor
```

建议属性语义：

```
openRatio
type: number
range: 0..1
persistent: true
replicated: true

targetOpenRatio
type: number
range: 0..1
persistent: false 或 true, 需要明确决定
replicated: true

motion
type: enum
values: IDLE, OPENING, CLOSING, BLOCKED
persistent: false
replicated: true
```

```
locked
type: boolean
persistent: true
replicated: true

enabled
type: boolean
persistent: true
replicated: true

lastInteractor
type: reference
persistent: false
replicated: optional
```

不要把这些字段重新硬编码进一个巨大 `DoorState` Record。

State Schema 应保持可数据化和可扩展。

九、任务五：Kinematic 模型接入运行时

Generation Pipeline 必须能输出动态部件元数据：

```
ComponentPath
ParentComponentPath
LocalTransform
Pivot
MotionDefinition
StateDriver
```

Door 建议结构：

```
door/root
├── frame
├── trim
└── panel
    ├── glass
    └── handle
```

动态根节点：

```
door/root/panel
```

运动定义：

```
movementType: ROTATE
axis: vertical
pivot: hinge edge
minAngle: 0
maxAngle: 90 degrees
stateDriver: openRatio
```

KinematicEvaluator 输入:

```
KinematicModel
RuntimeState
```

输出:

```
Map<ComponentPath, Transform3d>
```

必须接入三个消费者

1. Render

动态 Mesh Instance 使用当前 Transform。

禁止重建 Mesh。

2. Collision

第一版使用动态 OBB 或变换后的简单箱体。

3. Picking

鼠标射线检测 and 选择高亮必须使用动态姿态。

十、任务六：完成服务器权威网络协议

已有消息应尽量复用。

至少需要:

```
CommandRequestMessage
CommandAcceptedMessage
CommandRejectedMessage
ObjectSnapshotMessage
```



```
StateDeltaMessage
EventDeltaMessage
ObjectRemovedMessage
ObjectUnloadedMessage 或视距卸载语义
ObjectResyncRequest
```

CommandRequestMessage

至少包含：

```
protocolVersion
commandId
objectId
commandType
commandPayload
expectedObjectRevision
expectedStateRevision
actor
timestamp 或 client sequence
```

服务器验证：

```
对象存在
对象已加载
Command Type 允许
Payload 合法
Actor 有权限
Revision 匹配
Capability 可用
```

服务器拒绝时返回结构化错误码：

```
OBJECT_NOT_FOUND
OBJECT_NOT_LOADED
PERMISSION_DENIED
REVISION_CONFLICT
COMMAND_UNSUPPORTED
CAPABILITY_MISSING
STATE_INVALID
OBJECT_LOCKED
RATE_LIMITED
INTERNAL_ERROR
```

Resync

客户端检测以下情况时请求完整快照：

base revision 不匹配
state revision 跳号
event sequence 跳号
未知对象收到 delta
协议状态损坏

服务器返回：

ObjectSnapshotMessage

客户端用 Snapshot 替换本地 Replica。

幂等性

服务器必须记录最近处理的：

commandId

重复 Command Request 不得重复执行。

十一、任务七：修正版本体系

必须明确区分：

ReplicationProtocolVersion
SnapshotSchemaVersion
ArchitecturalObjectSchemaVersion
ArchitecturalTypeDefinitionVersion
StateSchemaVersion
BehaviorSchemaVersion

禁止在 Codec 中随意硬编码：

1

建立统一版本常量和迁移入口。

所有 Decode 必须检查版本。

不支持的版本返回结构化错误，不要只抛模糊异常。

十二、任务八：世界持久化闭环

`OpeningBlockEntity` 或对应平台实体不能成为业务逻辑中心。

建议流程：

加载

```
BlockEntity.load()
↓
decode ArchitecturalObjectSnapshot
↓
ArchitecturalRuntime.restore(snapshot)
↓
保存 objectId → session 映射
```

保存

```
BlockEntity.writeNbt()
↓
runtime.find(objectId)
↓
session.snapshot()
↓
encode NBT
```

卸载

```
Chunk unload
↓
snapshot
↓
persist
↓
runtime.unload(objectId)
```

删除

```
Block broken
↓
DeleteObjectCommand
↓
runtime.remove(objectId)
↓
replicate ObjectRemoved
↓
clear persistence
```

必须区分：

unload

与：

remove

十三、任务九：测试要求

本阶段不得只增加单元测试。

必须建立以下测试层级。

1. Runtime Unit Test

DoorRuntimeLifecycleTest

验证：

- 创建；
- 开门；
- Tick；
- 状态变化；
- Kinematics；
- Snapshot；
- Restore；
- Revision。

2. Transaction Test

RuntimeTransactionTest

验证：

- 原子提交；
- 错误 Revision 拒绝；
- State Schema 拒绝非法值；
- 失败时不产生部分状态；
- Event 与 Revision 一致。

3. Network Codec Test

验证所有允许消息的：

encode → decode → equals

并测试：

- 未知协议；
- 缺少字段；
- 超大消息；
- 非有限数字；
- 负 Revision；
- 不连续 Sequence。

4. Replica Test

DoorReplicationTest

验证：

- Snapshot；
- State Delta；
- Event Delta；
- 丢包；
- Revision conflict；
- Resync；
- 重复消息幂等。

5. Fabric GameTest

DoorWorldLifecycleGameTest

流程：

```
放置
→ 修改参数
→ 开到 60%
→ 保存
→ 卸载
→ 重载
→ 校验完整恢复
```

校验至少：

```
ObjectId
TypeId
ParameterSet
HostBinding
RuntimeState
ObjectRevision
StateRevision
Kinematic Pose
Geometry Fingerprint
```

6. Two-client Integration Test

测试：

```
Client A 开门
Server 执行
Client A 和 B 同步
Client B 丢一个 Delta
Client B 请求 Resync
最终状态一致
```

如果现有测试框架暂时无法真正启动两个 Minecraft Client，应先实现平台无关的双 Replica 测试，并在文档中明确真实双客户端 GameTest 仍未完成。

不要虚报完成度。

十四、任务十：架构检查

更新或增加 `checkArchitecture` 。

必须禁止：

```
aperture-runtime-model 导入 Minecraft
aperture-runtime-model 导入 Gson 或 NBT
Behavior 直接导入 Minecraft
CommandHandler 直接修改 RuntimeState
Fabric 直接修改权威状态
Client 发送 StateDelta
Runtime 依赖 ImGui
Render 触发 Geometry 编译
BlockEntity 创建第二套 Door State
```

建议加入源码模式检查：

```
".put(\"openRatio\")
\"setOpenRatio(\"
\"new DoorState(\"
```

如果这些模式出现在事务实现之外，应失败或人工审查。

十五、任务十一：清理模块依赖

当前通用 Runtime 不应长期依赖具体 `aperture-opening` Family。

完成 Door 垂直切片后，调整注册方式。

建议定义：

```
public interface ArchitecturalFamilyPlugin {
    void registerDefinitions(DefinitionRegistrar registrar);

    void registerCapabilities(CapabilityRegistrar registrar);

    void registerBehaviors(BehaviorRegistrar registrar);

    void registerCommandHandlers(CommandHandlerRegistrar registrar);

    void registerKinematics(KinematicRegistrar registrar);
}
```

实现：

```
OpeningFamilyPlugin
```

由 Bootstrap 组合：

ArchitecturalRuntime
不知道 Door

OpeningFamilyPlugin
注册 Door

依赖方向应逐步变为：

aperture-opening
↓
aperture-runtime
↓
aperture-runtime-model

而不是 Runtime 主模块直接依赖具体 Door 实现。

不要为了这项重构破坏已完成的垂直切片。

建议在所有世界生命周期测试通过后再做。

十六、文档更新

更新：

docs/architecture/14-k2-architectural-runtime-object.md
docs/architecture/platform/03-runtime-pipeline.md
docs/architecture/platform/04-runtime-environment.md
docs/architecture/07-serialization.md
docs/architecture/00-INDEX.md

不要只写目标架构。

每份文档明确区分：

Target Architecture
Implemented
Integrated
World Proven
Multiplayer Proven
Remaining Gaps

不要使用模糊百分比作为主要完成度。

推荐状态：

Documented
Implemented
Unit Proven
Platform Integrated
World Proven
Multiplayer Proven
Production Ready

十七、实施顺序

严格按照以下顺序推进。

Phase 1: 审计和运行时入口

1. 审计当前 K2 实现
2. 建立 ArchitecturalRuntime
3. 建立 RuntimeObjectSession
4. 建立 RuntimeObjectRepository
5. 添加 DoorRuntimeLifecycleTest

Phase 2: 事务统一

6. 建立 RuntimeMutation
7. 建立 RuntimeTransaction
8. 禁止所有直接 State 修改
9. 统一 Command → Behavior → Transaction
10. 添加 Revision 与原子性测试

Phase 3: 门的动态运行

11. 完善 Door State Schema
12. 完善 Door Behavior
13. Runtime Tick 推进 openRatio
14. Kinematic Transform 接入 Renderer
15. 动态碰撞
16. 动态 Picking

Phase 4: 持久化

17. BlockEntity 恢复 Runtime Session
18. BlockEntity 保存 Snapshot

19. 区分 unload 与 remove
20. 添加 Place → Save → Reload GameTest

Phase 5: 网络

21. CommandRequest
22. Server Validation
23. State/Event Delta
24. Revision Conflict
25. Resync
26. Duplicate Command Idempotency
27. 双 Replica 或双客户端测试

Phase 6: 解耦和文档

28. 通过 Family Plugin 解耦 Runtime 与 Opening
29. 更新架构检查
30. 更新文档
31. 清理废弃适配器
32. 运行完整测试

十八、提交拆分

每一个提交必须可编译并尽可能通过测试。

建议提交：

1. docs: audit K2 runtime vertical slice
2. feat(runtime): add ArchitecturalRuntime lifecycle
3. feat(runtime): add RuntimeObjectSession repository
4. feat(runtime): add transactional state mutation
5. refactor(runtime): route command and behavior through transactions
6. test(runtime): add door lifecycle and revision tests
7. feat(opening): complete door runtime state machine
8. feat(kinematic): bind door state to dynamic transforms
9. feat(render): apply kinematic transforms to dynamic parts
10. feat(collision): add dynamic panel collision
11. feat(interaction): update picking for dynamic components
12. feat(persistence): restore runtime sessions from block entities
13. test(fabric): add door world lifecycle GameTest
14. feat(network): add authoritative command request protocol
15. feat(network): add revision conflict and resync
16. test(replication): add replica loss and recovery tests
17. refactor(runtime): add architectural family plugin registration

- 18. chore: strengthen architecture checks
- 19. docs: update K2 implementation status

不要在一个提交中同时完成 Runtime、Network、Render 和 Editor。

十九、完成标准

只有以下条件全部满足，K2.2 才算完成：

- ☐ 一扇门可以在世界中创建。
- ☐ 门由 `ArchitecturalRuntime` 激活。
- ☐ Runtime Session 是唯一权威内存状态。
- ☐ 状态只能通过事务修改。
- ☐ 玩家交互转换为 Command。
- ☐ 服务器校验并执行 Command。
- ☐ Behavior 返回 Mutation，不直接改世界。
- ☐ Runtime Tick 推进开闭运动。
- ☐ 门扇不通过重新生成 Mesh 来运动。
- ☐ Render 使用动态 Transform。
- ☐ Collision 使用动态姿态。
- ☐ Picking 使用动态姿态。
- ☐ BlockEntity 保存 Snapshot。
- ☐ 世界重载后完整恢复。
- ☐ 客户端不能直接修改权威状态。
- ☐ 两个客户端看到一致状态。
- ☐ Revision 冲突能触发 Resync。
- ☐ 重复命令不会重复执行。
- ☐ Runtime 不直接依赖具体 Door Family。
- ☐ 核心 Runtime 测试无需启动 Minecraft。
- ☐ Fabric 世界生命周期测试通过。
- ☐ 文档准确描述实际实现。
- ☐ 不新增任何门窗 Family。

二十、完成后输出

完成每个阶段后，输出：

1. 修改摘要

新增文件
修改文件
删除文件
废弃文件

2. 架构变化

解释：

旧调用链
新调用链
消除的重复路径
保留的兼容适配器

3. 测试结果

列出：

测试名称
是否通过
验证内容
仍未执行的测试

4. 未完成问题

必须诚实说明：

- 尚未实现；
- 暂时使用简化实现；
- 需要真实 Minecraft 环境验证；
- 潜在性能问题；
- 网络边界问题。

5. 下一阶段建议

完成 K2.2 后，下一阶段才是：

K2.3 ImGui Editor Shell

但不要在本任务中提前开发大规模 ImGui UI。

最终原则

本阶段不是继续搭积木。

本阶段要证明所有已经存在的基础设施能够组成一个真实系统：

ArchitecturalObject



Generation



Runtime



Command



Behavior



State



Kinematics



Persistence



Replication

成功标准不是：

已经创建了很多接口和类。

成功标准是：

一扇门能够在真实 Minecraft 世界中被创建、放置、运行、交互、保存、重载和多人同步，并且整个过程只走 Aperture 的统一建筑运行时。